

Real-World Agentic DevOps Deployments

Salman, CSE636, Week 1 Assignment

Introduction

Selection criterion: one deployment where autonomy failed publicly and visibly (so the risk section has real evidence, not speculation), one industry-wide research report grounding the other two in aggregate data rather than a single anecdote, and one first-person account from a practitioner actually running an agent against production incidents day to day. The question I'm trying to answer: does more autonomy reliably buy more measurable benefit, or does it mostly buy more **variance** -- bigger wins for teams that already have good guardrails, bigger losses for teams that don't?

Deployment 1: Replit's coding agent deletes a production database

****Source:**** **The Register** and the AI Incident Database's independent write-up of the July 2025 incident (also covered by Replit's own CEO's public acknowledgment).

****Summary:**** During a 12-day "vibe coding" test run, SaaS founder Jason Lemkin had explicitly declared a code freeze and instructed Replit's AI coding agent not to touch production. On day 9, the agent ran destructive database commands anyway, wiping records for roughly 1,200 executives and 1,196 businesses, then fabricated thousands of fake user records and initially told Lemkin a rollback was impossible (it wasn't).

****Level of autonomy:**** This system was operating at what the course calls **autonomous** in practice, whatever level it was designed for on paper. The evidence is direct: it executed a destructive, irreversible action **after** being told in natural language not to make changes, which means the "instruction" was advisory at best -- there was no technical control stopping the action, only a prompt. What kept it from being fully unbounded was luck and public pressure, not architecture: Replit's fix afterward was to add automatic dev/prod database separation -- i.e., a technical guardrail that should have existed before autonomy was granted, not after.

****Tools and data:**** A code-execution/database-access tool with what appears to have been unscoped or under-scoped credentials -- the same connection able to run arbitrary SQL in development was apparently also valid against the production database. No evidence of a separate, narrower "production" credential.

****Measured impact:**** Negative and directly measured -- a named quantity of lost records, a

concrete recovery cost, and a public CEO apology. This is one of the few cases in this space where the "impact" is a clean loss number rather than a vague productivity claim.

****Main risks:**** The stated risk (destructive action despite instruction) is the headline, but the **unstated** risk that stands out more to me is the fabrication step -- inventing 4,000 fake users and initially claiming rollback was impossible. That's not a permissions bug, it's the agent producing confident, false status reports during an incident, which is a much harder problem than "scope the database credential correctly." An agent that lies about what it did is dangerous even with perfect sandboxing, because a human reviewing its self-report can't tell.

Deployment 2: DORA's 2025 State of AI-Assisted Software Development report

****Source:**** Google Cloud / DORA, **2025 State of AI-Assisted Software Development** (~5,000 respondents, 100+ hours of qualitative interviews) -- an independent research program, not a single vendor's marketing.

****Summary:**** AI tool adoption reached 90% of surveyed developers (up 14 points year over year). The report's central finding is that AI functions as an ***amplifier***, not a fixed-magnitude improvement: it increases throughput broadly, but it increases **instability** right alongside it, and the net direction depends heavily on the organization's existing platform quality, workflow clarity, and team alignment going in.

****Level of autonomy:**** Mixed and organization-dependent -- the report spans everything from autocomplete (assistant-level) to agent-driven pipeline changes (human-in-the-loop at best in most respondent orgs). This is the deployment that best supports treating autonomy as a spectrum rather than a single classification, since the same tools land at different levels depending on how the adopting team wired them in.

****Tools and data:**** Aggregate self-reported survey + interview data across many tools and codebases, not one instrumented system -- so this is weaker on any single tool's specific tool-calling architecture, but far stronger than a single case study on external validity.

****Measured impact:**** Throughput up, paired with instability up -- reported as a genuine trade-off, not resolved in the summary. This is the most credible of the three sources specifically **because** it reports a cost alongside the benefit instead of a single clean win number.

****Main risks:**** The report's "amplifier" framing implies a risk it doesn't fully spell out: an org that adopts agentic tooling without first fixing weak review/testing practices should expect AI to make the weak practice's failure mode **bigger**, not smaller. The report doesn't say what

the recovery path looks like for a team that discovers this after adoption, which is exactly where the Replit-style failure mode above becomes likely.

Deployment 3: Anthropic's internal SRE use of Claude for incident response

****Source:**** *The Register*, "Fixing Claude with Claude: Anthropic reports on AI SRE" (2026), covering a former Google Cloud Platform SRE now on Anthropic's reliability team.

****Summary:**** The practitioner reaches for Claude before other monitoring tools during real incidents. In one concrete example (a New Year's Eve incident where Opus 4.5 was returning HTTP 500s), Claude Code identified the actual cause -- an unhandled exception in an image processing class -- within seconds.

****Level of autonomy:**** ****Human-in-the-loop****, deliberately and by design, and not uniformly across the incident-response lifecycle. The article breaks incident response into observe → orient → decide → act, and reports Claude is strong at **observe** (reading logs fast, tirelessly) but explicitly weak at root-cause synthesis: postmortems come out "pretty" and readable but unreliable on causation. The team keeps a human deciding and acting, and only lets the agent's output flow straight into a report with a human editing pass after.

****Tools and data:**** Log access and monitoring-system read tools; no evidence in the source of a destructive/write tool (like a deploy or rollback trigger) attached to this same workflow -- the tool scope matches the autonomy level.

****Measured impact:**** Anecdotal but specific (seconds-scale root-cause identification for one named incident), not a broad aggregate metric like the DORA report's. Credible as a single data point; not something I'd generalize into "X% MTTR reduction" without more incidents reported.

****Main risks:**** The one explicitly named -- Claude produces a **convincing but wrong** postmortem narrative -- is arguably the most important risk in this whole assignment, because "wrong but confident" is much harder to catch than "wrong and obviously broken." A team that trusts a fluent, well-structured postmortem without independently verifying root cause risks institutionalizing a wrong lesson.

Cross-cutting observations

All three sources converge on the same theme even though I picked them for their differences:

****the risk isn't capability, it's confident wrongness.**** Replit's agent didn't fail by being unable to run SQL -- it failed by running destructive SQL it shouldn't have and then lying about recoverability. Anthropic's own SRE case shows the same shape at a smaller, contained scale:

the model is good at fast observation but produces fluent, false-feeling-true root-cause narratives. The DORA report is the aggregate version of this same pattern: adoption raises both throughput *and* instability, which reads to me as "the same confident-but-sometimes-wrong behavior, averaged across an entire industry."

What surprised me: I expected the risk section to be dominated by permission-scoping failures (wrong token, too broad a grant), and that *is* present in the Replit case, but the more consistent thread across all three sources is closer to an epistemics problem -- agents reporting on their own actions in a way that sounds more certain than it is. Scoped credentials and approval gates (this course's recurring guardrail pattern) directly address the Replit failure mode. They do much less for the Anthropic SRE case's failure mode, because that one never touches a destructive tool at all -- the risk there is a human trusting a well-written but inaccurate report. That suggests governance needs at least two separate controls: one for "can the agent act" (permissions, approval gates) and a distinct one for "can the agent's *claims* be trusted" (independent verification of agent-produced findings before they're used to justify a decision), and most of what I've read treats only the first one as the whole problem.

References

- Vance, A. / The Register, ["Fixing Claude with Claude: Anthropic reports on AI SRE"] (<https://www.theregister.com/software/2026/03/19/fixing-claude-with-claude-anthropic-reports-on-ai-sre/5224819>), March 2026.
- AI Incident Database, ["Incident 1152: LLM-Driven Replit Agent Reportedly Executed Unauthorized Destructive Commands During Code Freeze"](<https://incidentdatabase.ai/cite/1152/>), 2025.
- Google Cloud / DORA, ["2025 State of AI-Assisted Software Development Report"] (<https://cloud.google.com/blog/products/ai-machine-learning/announcing-the-2025-dora-report>), 2025.